2024 IEEE International Conference on Intelligent Signal Processing and Effective Communication Technologies (INSPECT) | 979-8-3503-7952-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/INSPECT63485.2024.10896014

Optimizing Job Matching Through Automated Resume Screening with NLP and Machine Learning

CH Sai Surya
2021IMT-024
Information Technology
ABV-IIITM Gwalior

GLVS Kiriti
2021IMT-035
Information Technology
ABV-IIITM Gwalior

K Satya Saketh
2021IMT-050
Information Technology
ABV-IIITM Gwalior

P Sri Charan
2021IMT-072
Information Technology
ABV-IIITM Gwalior

A. Anjali
Assistant Professor
Information Technology
ABV-IIITM Gwalior

***Abstract*—Resume screening determines a candidate's qualification for a position based on their education, experience, and other resume information. Candidates are only called for interviews if their resume matches the job description. Manual resume examination is time-consuming and burdensome, prompting companies to use applicant tracking systems to shortlist candidates based on their skills. The primary cause of rejection is often a mismatch between the job role and the applicant's skill set. Job seekers must understand which job categories align with their skill sets. This study employs natural language processing (NLP) and machine learning (ML) techniques to evaluate applicants in real time. Methods such as tokenization, stemming, lemmatization, TF-IDF, and string matching are used to identify keywords. Subsequently, ML models are applied to determine the most suitable role for each applicant. Four models—SVM, logistic regression, random forest, and k-nearest neighbors—are compared for accurate classification, with SVM outperforming the other benchmarks. This project aims to revolutionize the resume screening process by integrating advanced technologies and refining our methodologies.**

***Index Terms*—Machine Learning, Natural Language Processing, Tokenization, Lemmatization**

I. Introduction

In today's fast-paced job market, efficiently sifting through a large pool of applicants to find the most suitable candidates is crucial. When an employer posts a job opening, they often receive many applications, but around 75 percent do not possess the required expertise. Hiring managers must carefully evaluate each candidate's resume to select the most suitable candidate. The traditional method of manually reviewing every resume, ranking it, and finally picking top-ranked resumes is time-consuming and error-prone due to human limitations. Human resources departments may receive thousands of resumes and cover letters daily, complicating the process. The challenges a company faces in selecting the right candidate for their job are burdensome because of the high number of applicants. The recruiter's biggest task is to whittle down a vast candidate pool to the select few most qualified for the job.

To address this issue, we propose implementing a system that can efficiently analyze documents or resumes to quickly identify the required skill set and categorize candidates based on their skills, overcoming the limitations of manual sorting imposed by the company. Machine learning and Natural Language Processing are some strategies we plan to employ in the up-and-coming field of HR technology. Which aims to automate or streamline manual processes like resume screening. A single position can attract numerous applicants due to the presence of platforms such as LinkedIn, Twitter, and online talent databases. Automated systems are increasingly common in modern HR departments, facilitating the screening of resumes and enhancing efficiency. This enables HR professionals to dedicate more time to meaningful interactions with both applicants and employees, i.e., instead of spending a lot of time filtering the best resumes, they can use Automated systems for filtering resumes and spend more time conducting further rounds of interviews for the applicants whose resumes are filtered by the automated system.

Our Model uses NLP (Natural Language Processing) to screen the resumes efficiently. Natural Language Processing (NLP) is a machine-learning technology that helps computers explain, manipulate, and recognize human language. Modern organizations accumulate vast amounts of voice and text data from different channels such as emails, text messages, social media platforms, newsfeeds, videos, and audio. Utilizing NLP software allows them to automatically analyze and extract meaning or sentiment from this data and promptly respond to human communication in real time.

When the employer uploads the applicant's resume to this website. First, it extracts all the test/data present in the resume. The text from the resume file is extracted at the application's front end. We further give the resume (i.e., text extracted from the resume file) as input to our machine-learning model. The first step is cleaning the given resume, i.e., we remove the URLs, hashtags, mentions, special characters, and punctuation, which is a cleaned resume. The second step is converting the cleaned resume into a vector form using TF-IDF. Term Frequency-Inverse Document Frequency (TF-IDF) is the most commonly used statistical method in natural language processing and information retrieval. It assesses the significance of a term in a document compared to a collection of documents (i.e., a corpus). Words in a text document are converted into numerical values that reflect their importance through a text vectorization process. There are various scoring schemes for text vectorization, with TF-IDF being one of the most prevalent. Now, we feed this vector as the input to the Machine Learning Model, which classifies the

979-8-3503-7952-5/24/$31©2024 IEEE

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on February 05,2026 at 08:33:42 UTC from IEEE Xplore. Restrictions apply.

resumes and provides a better fit suited for the given resume. For classification, we are using KNN (K-nearest neighbors).

The KNN algorithm is a widely used method in machine learning for classification and regression problems. In KNN, first, we find the nearest neighbors of a particular data point using any distance metric like Euclidean Distance. Now, after finding the nearest neighbors, if the problem is a classification problem, we see the most common category among all the categories of its K neighbors and assign that particular category to that particular data point. If the problem is a regression problem, then we find the average or the weighted average of its k neighbors to find the output at a particular data point. Generally, we take odd values of k to avoid ties between two categories. There is no particular way to choose the value of k; we check some values of k and conclude by choosing a particular value of k.

II. Literature Survey

Resume screening is critical for organizations to identify qualified candidates from a large applicant pool for open positions. With the rise of digital resumes and increasing job applications, manual resume screening has become highly time-consuming and inefficient. Various methods have been proposed, ranging from rule-based systems to advanced neural networks and ensemble models. This literature review aims to overview existing research on automated resume screening, covering the key algorithms, features, and evaluation metrics employed.

”Screening and Ranking Resumes using Stacked Model” [1] is authored by Rasika Ransing, Nikita Bhrugumaharshi Emberi, Akshaya Mohan, and Kailas Mahavarkar. The paper presents an automated resume screening application that utilizes Machine Learning algorithms such as KNN, Linear SVC, and XGBoost. A two-level stacked model combining these algorithms is constructed to predict specific job profiles accurately from a text description. The framework aims to assist organizations in shortlisting candidates and enable job applicants to assess the suitability of their resumes for specific roles. Additionally, a ranking system is implemented to feature the most relevant profiles at the top for employers.

”Resume Classification using Elite Bag-of-Words Approach” [2] is authored by Muskan Sharma, Gargi Choudhary, and Seba Susan. This paper explores using a recently introduced text vectorization technique called Elite bag-of-words for resume classification into different job profiles. The Elite bag-of-words method involves extracting the most significant keywords from each resume class using maximum entropy partitioning and concatenating these keywords across classes to form the final feature set. The experimental results demonstrate that the Elite bag-of-words approach outperforms the other methods for resume classification on a benchmark dataset.

”Resume Classification using Support Vector Machine” [3] and is authored by Gopal Kamineni, Kandula Akhil Sai, and Dr. G. Siva Nageswara Rao. The paper proposes an intelligent system to automate the tasks of resume analysis and classification using machine learning techniques, specifically the Support Vector Machine (SVM) algorithm. The system aims to take resumes and job descriptions as input and classify whether the resumes are suitable for the job based on parameters such as education, skills, work experience, and certifications. The authors discuss the limitations of existing manual and automated resume screening methods and propose a hybrid model that incorporates techniques from various approaches to improve accuracy.

III. Methodology

The high-level overview of our application is shown in Figure 1. First, the applicant or an employer uploads the resume file to the website. Then, in the front end, the application will extract all text from the resume file, referred to as a resume, in this paper. Before giving the data as input to the ML model, we should preprocess the data to make the task easier.

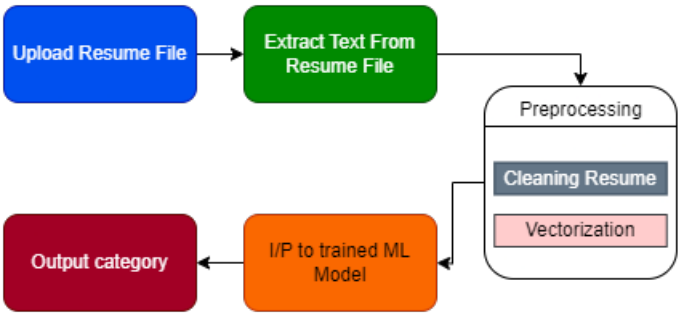


Fig. 1. High-Level Overview Of the application

Preprocessing is important because it cleans and organizes the data, making it suitable for performing logic by the model. In our application, the steps involved in preprocessing are cleaning the resume and vectorization.

A. *Preprocessing*

*1) **Cleaning Resume**:* It is not necessary to analyze the whole resume to predict the category. So, we removed some parts of the resume that do not affect the output, whether it is there or not. Cleaning a resume involves removing several types of entities in the resume.

Removing URLs: People usually add URLs to their resumes to add information about themselves. But considering just the URL phrases does not give any information about the category. So, we remove all the URLs present in the resume.

Removing Hashtags: We must eliminate hashtags to clean the resume further. Generally, hashtags contain # symbols and some alphanumeric characters but never have white space.

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on February 05,2026 at 08:33:42 UTC from IEEE Xplore. Restrictions apply.

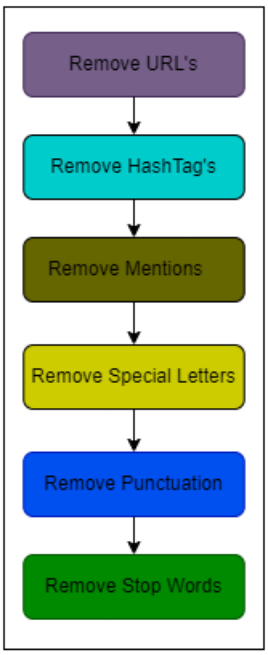


Fig. 2. Steps involved in cleaning of resume

Removing mentions: This phase is activated when handling text data from social media, such as Tweets or mail IDs. In most NLP tasks, mentions are considered unwanted since they don't generalize across samples. Thus, it's preferable to get rid of these.

Removing Special Characters: Non-alphanumeric characters are known as special characters. These characters are unique. These characters cause noise in algorithms and offer no benefit to text interpretation in most NLP jobs. Some examples of special characters are $,%,&,etc...

Removing Punctuations: Punctuations like commas, full-stop, etc.. don't represent any information, so we remove them. We replace punctuation with empty.

Removing stopwords: Some stopwords do not add any information. So we remove them. Stop words are basically words that are regularly used in many strings. These stop words are stored in our database so that we can remove the stop words quickly.

*2) **Vectorization**:* After cleaning the resume, each cleaned resume contains some words. Now, we should convert every cleaned resume into a vector representation, which we give as input to the ML model. We need to convert the cleaned resume to vectors because the model cannot take alphabets as input. we need to give numbers as the input. for converting the strings into vectors, there are different algorithms; before going into that, we need to know some important topics in NLP.

Tokenization: Tokenization is a method that converts paragraphs into sentences and sentences into words or tokens. while performing tokenization, we can divide the paragraph/sentences based on white space, punctuation, a special character, etc. So after tokenizing we have a list of tokens on which further NLP tasks are performed.

Stemming: Stemming is a method that reduces each word into its root form. It performs this process by removing suffixes and prefixes from a word and obtaining their core and base meaning. For example, runner, runs, and running are all reduced to one root word: run.

Lemmatization: Lemmatization is also a method that is mostly similar to stemming. However, the difference is that it reduces words into a base form or lemma by looking up a dictionary. So that even if it is slow it is more accurate than stemming.

The above methods extract important information from a text or strings before converting them to vectors. For vectorization, there are many algorithms; one of the most common algorithms is TF-IDF(Term Frequency - Inverse Document Frequency)

TF-IDF: Basically it is an algorithm that converts a text or strings of data into numerical vector forms. TF-IDF is used for several reasons, like measuring the importance of a term in a sentence relative to a corpus by calculating the frequency of that term. Formula (1) depicts the term frequency of a word that appears in a resume. To calculate Inverse Document Frequency, we use formula (2) in which df is document frequency, which means how many documents the term presents, and N is the count of documents or resumes. Finally, the TF-IDF score for a term in a document is calculated using the formula (3).

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d} \quad (1)$$

$$IDF(t) = \log(\frac{N}{1+df}) \quad (2)$$

$$TF - IDF(t,d) = TF(t,d) * IDF(t) \quad (3)$$

Fig 3 shows the flow chart of our vectorization process. First, after cleaning the resume (Mentioned in 1st step of pre-processing), we do tokenization on that cleaned resume, and then we get tokens. We do lemmatization again, and we join all the lemmatized tokens. So, the next step in our application is to use this TF-IDF algorithm, and we convert every cleaned resume into vector form. After doing TF-IDF, we get a sparse matrix as an output, referred to as vector form, and this process is called vectorization. Now, each cleaned resume is represented in a vector form or sparse matrix, i.e., it contains numerical values of some terms, which are TF-IDF scores of that term. Now, we give these vectors as input to our trained machine-learning model to predict the category.

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on February 05,2026 at 08:33:42 UTC from IEEE Xplore. Restrictions apply.

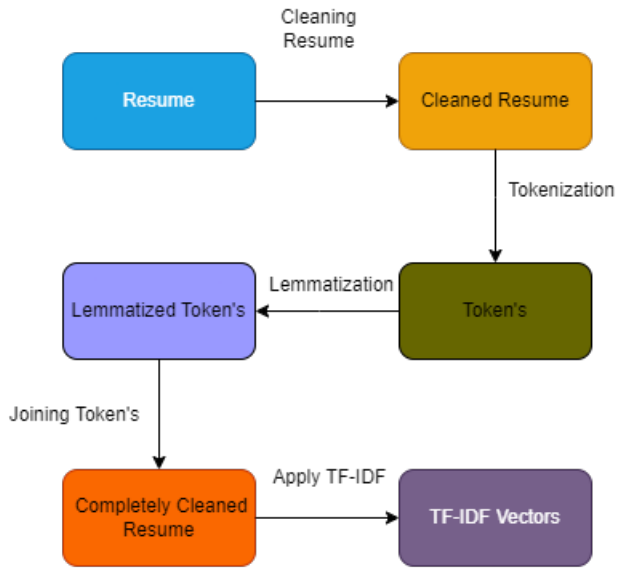


Fig. 3. flow chart representation of complete preprocessing

B. *Model Training*

There are many models that we can use for training the model; some of them are used in our work is listed below:

KNN(K-Nearest Neighbour): [4] The KNN algorithm is a widely used method in machine learning for classification and regression problems. In KNN, first, we find the nearest neighbors of a particular data point using any distance metric like Euclidean Distance. Now, after finding the nearest neighbors, if the problem is a classification problem, we see the most common category among all the categories of its K neighbors and assign that particular category to that particular data point. If the problem is a regression problem, we find the average or the weighted average of its k neighbors to find the output at a particular data point. Generally, we take odd values of k to avoid ties between two categories. There is no particular way to choose the value of k; we check some values of k and conclude by choosing a particular value of k.

Random Forest: [5]It is a supervised Machine-learning algorithm that can be used in classification and regression tasks. It overcomes the problem of overfitting. In a random forest, we take random samples from the given data set (Repetition is allowed), and then we make a randomized decision tree for every sample. based on that decision tree, each one generates an output; the final output is the majority of all the outputs. This is for the classification tasks, and the regression tasks, we take the average of all the outputs of every sample, and that will be the final output.

SVM (support vector machine): [6] SVM is a supervised machine learning algorithm for classification problems. In SVM, we draw a hyperplane that separates the different categories; the hyperplane depends on the no of features; if the number of features is 2, then the hyperplane is a line, and if the no of features is 3, then the hyperplane similarly is a plane, etc. The hyperplane is drawn in such a way that it maximizes the distance between the two closest points of different classes to the hyperplane

Logistic Regression: [7] Logistic Regression is a machine learning algorithm that belongs to the class of supervised machine learning. In Logistic Regression, a logistic function maps the input features with a value between 0 and 1. This is for binary classification here; we need multi-classification, so we use one vs rest classification where we consider one category as one entity and the other 24 categories as one entity and train the model in this way, we train 25 models, and when a new input is given we get a vector with 25 values and the corresponding category for that input is the corresponding model with highest probability value in that vector.

IV. Dataset Analysis

There are a total of 963 rows in the dataset. Our dataset consists of two columns; the first is the category for a particular resume. The second column consists of the corresponding resume, whose category is predicted in the first column. Firstly, we clean this resume and convert the cleaned resume into the vector using TF-IDF (Term Frequency-Inverse Term Frequency) and give this as an input to the ML model. The model gives the corresponding category suited for that resume as the output. We are using 80% of our dataset's data to train the model. We do the testing for the model on this trained model and with the remaining 20% of the data in our dataset. Our model classified all the resumes into different categories found the percentage of each category in all the resumes and represented it in the form of a pie chart shown in Figure 4.

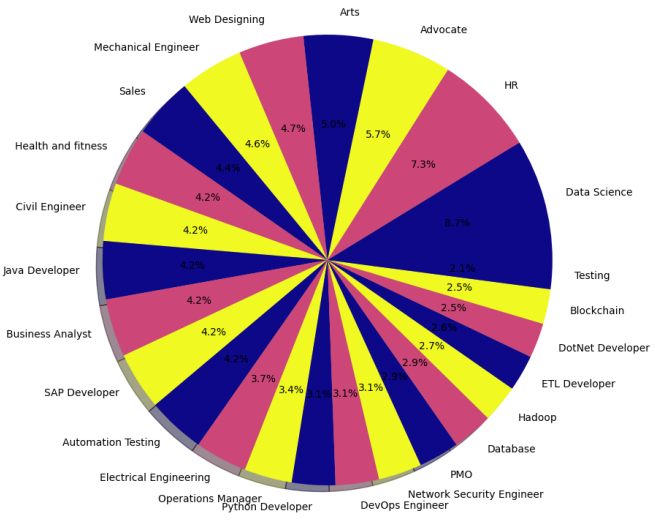


Fig. 4. pie chart representing the dataset

V. Experimental Results

One important thing we do when training a model is to assign a unique numerical value for each of the unique

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on February 05,2026 at 08:33:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ACCURACY ON TRAINING DATASET

S.No.	Algorithm	Training Data Accuracy
1	KNN	0.98829
2	Logistic Regression	0.99739
3	Support Vector Machine	1.0
4	Random Forest	1.0

TABLE II
ACCURACY ON TESTING DATASET

S.No.	Algorithm	Testing Data Accuracy
1	KNN	0.98445
2	Logistic Regression	0.99481
3	Support Vector Machine	0.99481
4	Random Forest	0.98445

categories present in our dataset. As in our database, there are 25 unique categories, so we assign a unique number between 0 to 24. Finally, the trained model will give any number between 0 and 24 as an output. So, according to that output, we check the mapping and find the predicted category. Table I shows the numerical values assigned to each category in our experimental outcome. After converting the cleaned resume to vector form, we give it as input to our trained model, and the below figures from Fig 5 to Fig 8 show the actual vs. predicted values when we train a model with different algorithms, as discussed previously. In Fig 5,6,7,8, we plotted the graph's actual vs. predicted values for each algorithm KNN, Random Forest, Logistic Regression, and SVM algorithms. In each graph, the actual value is represented by a blue dot; with a red dot, we represent a predicted value by the algorithm. Each graph's X-axis is the Data Point Index, and the Y-axis is the Value.

In each plotted graph, the Data Point Index means the index of a data point in any dataset. Here is our test dataset. Where Value means - It is the output of each data point. Here, we are representing both actual and predicted outputs. So now if many red dots and blue dots are overlapped we can deduce that algorithm is predicting values with high accuracy. In all figures, there are more red dots and very few blue dots because many red dots and blue dots are overlapping, i.e., both predicted and actual values are the same. But in Fig 5, i.e., for the KNN model, we can see 3 blue dots mean for 3 data points, there is a slight variation of predicted value from the actual value. We can see just one blue dot for the remaining models in Fig 6,7,8. So, by looking at the plotted graph, we can say that KNN has the least accuracy in predicting values.

Fig 9 shows the Front-end user interface of our application where an employer or an applicant can upload his resume file. So, we extract the text inside the resume file, give it as input to our model, and predict the output category. Table II shows the training data accuracy for each algorithm we used, and Table III shows the testing data accuracy for each algorithm used. Training data accuracy means we have the actual values for the training dataset. Now, we find the predicted values for the same training data set with different models and find the accuracy by identifying how many data points predicted

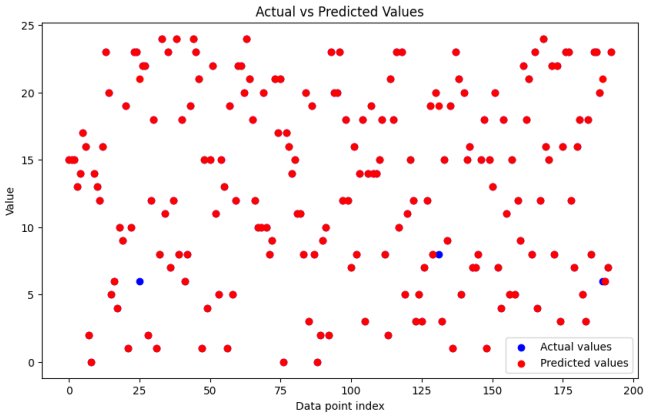


Fig. 5. Actual vs Predicted values for KNN

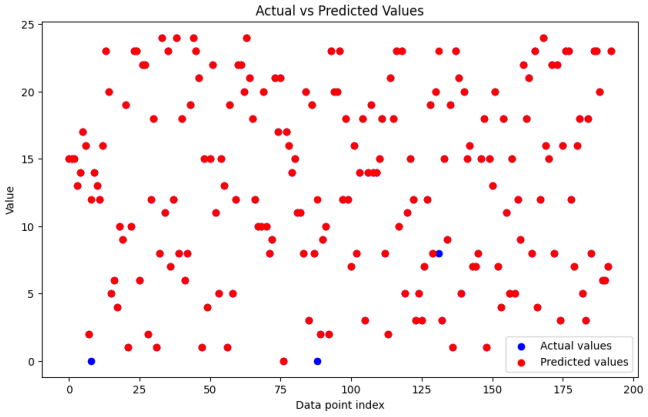


Fig. 6. Actual vs Predicted values for Random Forest

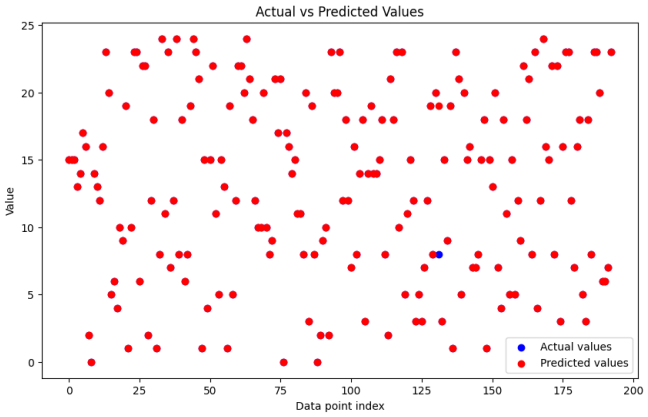


Fig. 7. Actual vs Predicted values for Logistic Regression

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on February 05,2026 at 08:33:42 UTC from IEEE Xplore. Restrictions apply.

TABLE III
F1 SCORES FOR EACH MODEL

S.No.	Algorithm	f1 score
1	KNN	0.98381
2	Logistic Regression	0.99489
3	Support Vector Machine	0.99489
4	Random Forest	0.98234

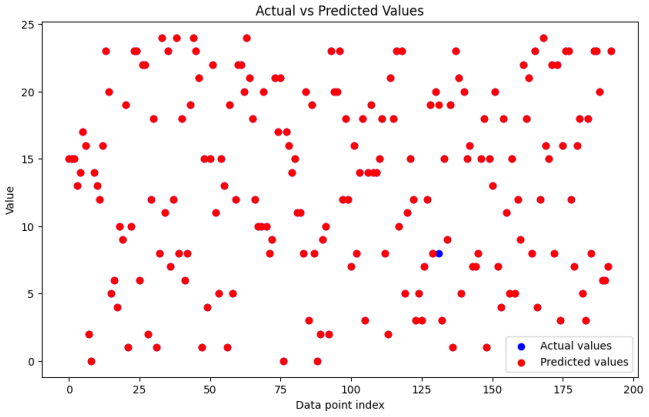


Fig. 8. Actual vs Predicted values for SVM

TABLE IV
NUMERICAL VALUES ASSIGNED FOR EACH CATEGORY

S.No.	Category	Number
1	Java Developer	15
2	Testing	23
3	DevOps Engineer	8
4	Python Developer	20
5	Web Designing	24
6	HR	12
7	Hadoop	13
8	Blockchain	3
9	ETL Developer	10
10	Operations Manager	18
11	Data Science	6
12	Sales	22
13	Mechanical Engineer	16
14	Arts	1
15	Database	7
16	Electrical Engineering	11
17	Health and fitness	14
18	PMO	19
19	Business Analyst	4
20	DotNet Developer	9
21	Automation Testing	2
22	Network Security Engineer	17
23	SAP Developer	21
24	Civil Engineer	5
25	Advocate	0

values deviate from the actual values in the training data set. Similarly, we do for the testing data set shown in Table III. Table IV shows the f1 score of each algorithm used to train the model. The higher the f1 score, the higher the performance and precision of that algorithm.

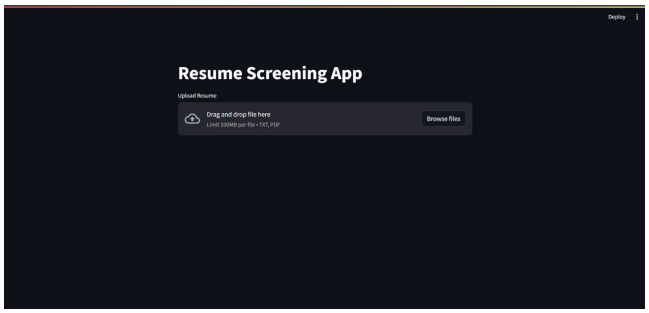


Fig. 9. UI of the frontened

VI. CONCLUSION

This study compared the accuracy of four algorithms—k-nearest neighbors (KNN), random forest, logistic regression, and support vector machine (SVM)—on our dataset for resume screening. The results, detailed in Tables 1 and 2, indicate that SVM and random forest achieved the highest accuracy for the training dataset. Logistic regression followed, with KNN producing the lowest accuracy among the four algorithms for the training data. SVM and logistic regression outperformed the other models for the testing dataset, followed by KNN and random forest. Overall, SVM demonstrated the best accuracy across training and testing datasets, underscoring its potential for enhancing the efficiency and reliability of the resume screening process. This project aims to revolutionize resume screening by integrating these advanced technologies and refining our methodologies.

REFERENCES

[1] R. Ransing, A. Mohan, N. B. Emberi, and K. Mahavarkar, "Screening and ranking resumes using stacked model," in *2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT)*, pp. 643–648, IEEE, 2021.

[2] M. Sharma, G. Choudhary, and S. Susan, "Resume classification using elite bag-of-words approach," in *2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT)*, pp. 1409–1413, IEEE, 2023.

[3] G. Kamineni, K. A. Sai, and G. S. N. Rao, "Resume classification using support vector machine," in *2023 3rd International Conference on Pervasive Computing and Social Networking (ICPCSN)*, pp. 91–96, IEEE, 2023.

[4] S. Zhang, X. Li, M. Zong, X. Zhu, and R. Wang, "Efficient knn classification with different numbers of nearest neighbors," *IEEE transactions on neural networks and learning systems*, vol. 29, no. 5, pp. 1774–1785, 2017.

[5] V. Y. Kulkarni, "Effective learning and classification using random forest algorithm," 2014.

[6] S. Yue, P. Li, and P. Hao, "Svm classification: Its contents and challenges," *Applied Mathematics-A Journal of Chinese Universities*, vol. 18, pp. 332–342, 2003.

[7] E. Bisong and E. Bisong, "Logistic regression," *Building machine learning and deep learning models on google cloud platform: A comprehensive guide for beginners*, pp. 243–250, 2019.

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on February 05,2026 at 08:33:42 UTC from IEEE Xplore. Restrictions apply.